

PYTHON

Présenté par :
Xavier TABUTEAU

La quasi-totalité des applications écrites aujourd'hui utilisent des données d'une manière ou d'une autre, ces données sont généralement fournies ou consommées par un service web. Les données sur Internet sont généralement présentées dans l'un des deux principaux formats suivants : XML et JSON.

- **XML**

- Extensible Markup Language
- Le XML est très similaire au HTML, mais avec quelques modifications pour l'aider à représenter des types de données plus génériques que le simple contenu Web.
- Les règles de structure du XML sont beaucoup plus strictes.
- Riche et expressif mais plus complexe que le JSON.
- <https://www.w3.org/XML>

- **JSON**

- JavaScript Object Notation
- Il s'agit essentiellement d'un moyen de prendre des objets arbitraires contenant des données et de représenter ces données dans un format textuel très concis.
- JSON est à l'origine dérivé de JavaScript, mais il est désormais pris en charge par un large éventail de langages de programmation.
- Compact et Léger, facile à écrire, lire et traiter.
- <https://www.json.org/>

Les deux formats ont en commun le fait qu'ils sont indépendants de la plate-forme ou du langage de programmation dans lequel ils sont utilisés.

Web

Web

• Règle de base du formatage XML

- Les fichiers XML doivent toujours avoir une seule balise racine qui contient toutes les autres balises.
- Les fichiers XML peuvent contenir une déclaration de document facultative.
- Les balises vides, c'est-à-dire les balises qui ne contiennent aucune donnée, doivent être fermées par une barre oblique à l'intérieur du nom de la balise : `<nom_balise/>`
- Les balises XML qui possèdent des attributs doivent se voir attribuer des valeurs et ces dernières doivent être placées entre guillemets.
- Les balises doivent être correctement imbriquées les unes dans les autres.
- La spécification XML réserve les attributs et les noms de balises qui commencent par les caractères XML.

Remarque : Le XML n'impose aucun type de données.

```
<sandwich>
  <name>Reuben</name>
  <bread type="rye" toasted="true"/>
  <contents>
    <item name="corned beef" />
    <item name="sauerkraut" />
    <item name="Swiss cheese" />
    <dressing name="Russian" />
  </contents>
  <side-item name="pickle" />
  <price currency="usd">8.99</price>
</sandwich>
```

Web

Web

- Règle de base du formatage JSON

Number	Nombre décimal, il ne fait pas de distinction entre un entier et un float.
String	Chaîne de caractère délimité par des guillemets.
Boolean	Valeur True ou False.
Null	Valeur nulle.
Array	Liste des valeurs ordonnées. Les sets et les tuples n'existent pas en JSON.
Object	Collection d'objets de type clé-valeur.

```
{
  "filename" : "JSON example file",
  "version" : 2.0,
  "is_example" : true,
  "linked_file" : null,
  "array" : [1, 2, 3.0, "4"],
  "obj" : {
    "value1" : 1,
    "value2" : ["a", "b", "c"]
  }
}
```


Web

Web

- **Module Python pour le web**

Il existe plusieurs modules Python différents pour travailler avec des données Internet. Sur la documentation Python, Il existe des modules permettant de travailler avec des navigateurs et des serveurs Web, de manipuler des URL, de travailler avec des protocoles de transport tels que HTTP et FTP, de manipuler des courriels. Les formats standards sont fournis avec Python. Pour les formats propriétaires il faut télécharger les modules spécifiques.

<https://docs.python.org/3/library/internet.html>

Urllib	Contient plusieurs sous-modules liés au travail avec des données Internet. Tels que l'émission de requêtes et la réception de réponses d'URLs. Il gère les erreurs et les exceptions au cours de ce processus et analyse les informations relatives aux URL.
Http	Contient du code pour travailler avec les cookies, les serveurs et d'autres protocoles HTTP de bas niveau.
Json	Permet de convertir des données Python en JSON et des données JSON en types de données Python natifs.
XML	Python dispose de plusieurs modules différents pour travailler avec le XML. On peut travailler à l'aide des API SAX et DOM, ainsi que de l'API ElementTree, qui offre un moyen plus pratique de travailler avec des données XML.

Web

• URL Lib

Web

La bibliothèque standard de Python est livrée avec un ensemble intégré de modules permettant d'accéder aux données Internet, contenus dans le paquet urllib. Le paquet urllib contient quatre modules :

Urllib.request	Gère l'ouverture et la lecture des url.
Urllib.error	Définit les classes d'exceptions pour les erreurs soulevées par le module request.
Urllib.parse	Pour analyser les structures url.
Urllib.robotparser	Pour travailler avec les fichiers robots.txt.

Le module urllib est relativement simple et facile à utiliser, mais il présente plusieurs inconvénients par rapport aux autres bibliothèques disponibles.

Urllib ne supporte qu'un sous-ensemble de l'ensemble des opérations HTTP, telles que get et post à moins que vous ne les implémentiez vous-même.

Urllib ne décode pas automatiquement les données retournées pour vous.

Il n'y a pas de fonctionnalités intégrées à urllib pour travailler avec certaines constructions de données web communes comme les cookies, l'authentification ou les sessions. Il faudrait importer d'autres modules.

<https://httpbin.org> permet de tester des requêtes HTTP comme get, post, etc...

urllib.py
urllibdata.py
urlliberr.py

Web

Web

• Requests

La bibliothèque Requests simplifie et améliore considérablement les modules standards de Python.

<https://docs.python-requests.org/en/latest/>

```
pip install requests
```

- Requests fournit une API très simple pour travailler avec diverses opérations HTTP. Chaque opération HTTP correspond en fait à un nom de méthode dans la bibliothèque Requests.
- Requests rend facile l'envoi des données supplémentaires au serveur, comme des paramètres ou des informations d'en-tête.
- Requests decode automatiquement les informations renvoyées.
- Requests decode du JSON et le convertit en objets Python directement.
- Requests détecte et gère également les erreurs et les redirections.
- Requests est fournie avec un ensemble de codes de réponse HTTP intégrés.
- Requests prend également en charge des fonctionnalités plus avancées comme l'authentification, les objets de session et même le téléchargement de données en continu.

```
requests.py  
reqerrs.py  
reqauth.py
```

- **Comment manipuler du JSON**

Le module JSON fait partie de la bibliothèque standard Python intégrée et contient des méthodes pour travailler avec le format.

Il existe deux méthodes pour convertir le texte en JSON :

Load	<i>Obj = load(file)</i>	Charge JSON directement depuis un fichier.
Loads	<i>Obj = loads(string)</i>	Travaille directement sur une chaîne de texte.

Il existe également deux méthodes pour prendre des objets et les sérialiser directement en JSON :

Dump	<i>Obj = dump(file)</i>	Ecrit le JSON dans un fichier.
Dumps	<i>Obj = dumps(string)</i>	Ecrit le JSON dans une chaîne de caractère.

Web

- Conversion JSON en objet Python

JSON Data	Python Object
object	dict
array	list
string	str
Integer number	int
Floating point number	float
true, false	True, False
null	None

- S rialisation des objets python en JSON

Python Object	JSON Representation
dict	object
list, tuple	array
str	string
int, long, float, Enums	number
True	true
False	false
None	null

json_req.py
json_parse.py
json_serialize.py
json_err.py

Web

Web

Web

• Comment travailler avec des données XML en Python

Les analyseurs XML fonctionnent essentiellement de deux manières. La première méthode est connue sous le nom de Simple API for XML (SAX). La deuxième méthode est appelée le Document Object Model (DOM).

Un analyseur syntaxique qui utilise la méthode SAX fonctionne comme une chaîne de montage. Il lit les données XML un caractère à la fois séquentiellement jusqu'à ce que le fichier entier ait été lu. Pendant la lecture des données XML, l'analyseur génère divers événements qui correspondent au contenu du XML. Votre application Python définit donc une classe qui traitera ces événements au fur et à mesure de leur arrivée.

Avantages :

- Econome en mémoire puisque le document entier n'a pas besoin d'être chargé en mémoire.
- Rapides, puisque votre application ne répond qu'aux événements qui l'intéressent réellement.
- Faciles à mettre en œuvre.

Désavantages:

- Aucun moyen d'accéder au contenu du XML dans n'importe quel ordre.
- Ne conserve pas d'informations contextuelles.
- Ne peut pas modifier les données XML au cours de leur traitement.

Web

- SAX

<https://docs.python.org/3/library/xml.dom.html>

Import xml.sax	L'API SAX en Python, est implémentée dans le module xml.sax.
Xml.sax.parse(file, handler)	Peut être utilisé si le XML provient d'un fichier.
Xml.sax.parseString(string, handler)	Peut être utilisé si le XML provient d'une variable de type chaîne des caractères.
Class xml.sax.ContentHandler	Classe prédéfinie qui possède des fonctions permettant de gérer le début et la fin du document, le début et la fin des balises ainsi que les données textuelles. En créant une sous classe héritant de cette classe, on peut remplacer simplement ces fonctions pour gérer chaque type de contenu et créer sa propre logique de traitement du contenu.

Web

sax_parse.py

Web

- DOM

Web

L'une des principales différences entre DOM et SAX est que le DOM vous permet d'accéder à n'importe quelle partie du fichier XML de manière aléatoire et facilite la modification du contenu du fichier. Lorsque vous utilisez le DOM pour analyser le code XML, il est lu en mémoire dans son intégralité et représenté sous forme d'arborescence. Vous pouvez ensuite utiliser diverses API pour travailler sur l'arbre documentaire résultant. Attention à la mémoire si le fichier est volumineux.

<https://docs.python.org/3/library/xml.dom.minidom.html>

dom_parse.py

Import xml.dom.minidom	Bibliothèque standard Python fournit une implémentation de l'API DOM.
Xml.dom.minidom.parseString(string, handler)	Peut être utilisé pour créer une structure arborescente de documents à partir du texte XML source. Pour ensuite être en mesure d'utiliser les éléments repris ci-dessous.
<i>getElementById</i> ou <i>getElementsByTagName</i>	Récupérer des éléments par ID ou par leur tagName.
<i>getAttribute</i> ou <i>setAttribute</i>	Récupérer ou définir les valeurs des attributs.
<i>createElement</i> ou <i>createTextNode</i> ou <i>appendChild</i>	Créer ou insérer un nouveau contenu.

Web

- **ElementTree**

L' API ElementTree se concentre sur une manière plus simple et plus efficace de travailler avec XML et cela conduit à quelques différences importantes par rapport à DOM.

- Les éléments sont traités comme des listes.
- Les attributs sont traités comme des dictionnaires.
- ElementTree API fournit les fonctions `defined` et `findAll` qui rend la recherche de contenu dans le XML très simple.

Ex: `Elem.findAll(queryExpression)`

<https://docs.python.org/3/library/xml.etree.elementtree.html>

`pip install lxml`

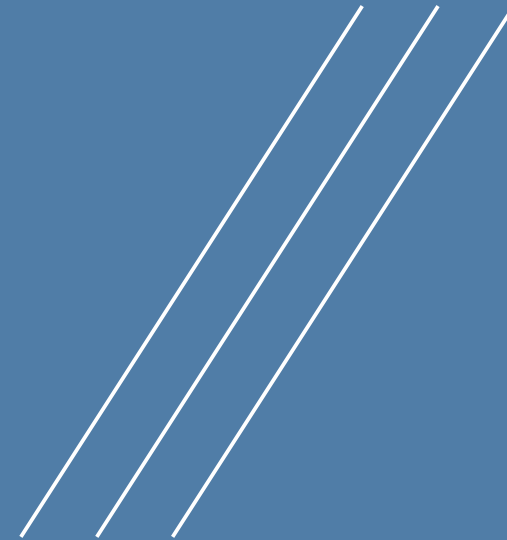
ou à partir d'une version compilée que vous pouvez télécharger sur

<http://www.lfd.uci.edu/~gohlke/pythonlibs/#lxml> puis :

`pip install c:\chemin_complet\nom_fichier_complet`

Query Expression	Description
Tagname	Trouver les éléments enfants immédiats avec ce nom.
//tagname	Trouver l'ensemble des éléments dans le document en entier.
Tagname[attr]	Rechercher les balises qui ont des attributs spécifiques.
Tagname[attr = val]	Rechercher les balises qui ont des attributs avec des valeurs spécifiques.

PYTHON



Présenté par
Xavier TABUTEAU